

AI-HDL Design Phase 1 (DP#1): Detailed Project Run & Technical Walkthrough

Video Source: DP#1 Example Project Run

Session Lead: Harish Kumar

Objective: Execute the full "RTL-to-GDSII" flow for the first design challenge, utilizing the [TinyQV](#) RISC-V core and the [OpenLANE](#) physical design toolchain.

1. Prerequisites & Directory Structure

The video begins with a pre-configured environment. To follow along exactly, your directory structure should resemble the following before starting:

```
~/AI-HDL/
├── tinyqv/                # The base RISC-V CPU repository
├── tinyqv-byte-peripheral-template/ # The template for your assignment
├── yosys/                 # Installed Synthesis Tool
├── OpenLane/             # Installed Physical Design Tool
└── designs/              # (Optional) Working directory
```

2. Phase 1: Local Synthesis (The Sanity Check)

Goal: Verify that your Verilog code is functionally correct and synthesizable *before* attempting the time-consuming physical design run.

Step 2.1: Setting up the Workspace

The speaker creates a dedicated directory for this specific attempt to keep the environment clean.

- **Command:**

```
mkdir DP_1
cd DP_1
```

Step 2.2: Aggregating the Source Files

You must gather the CPU core files, your peripheral template, and the verification harness into one place. The speaker executes a series of copy (`cp`) commands.

1. Copy the Peripheral Template:

- *Context:* You are using the "Byte Peripheral" template.
- **Command:**

```
cp -r ~/AI-HDL/tinyqv-byte-peripheral-template/* .
```

2. Copy the CPU Core Files:

- *Context:* The CPU files are in the `tinyqv/src` folder.
- **Command:**

```
cp -r ~/AI-HDL/tinyqv/src/* .
```

3. Copy Test & Verification Harnesses:

- *Context:* Needed for simulation (though strictly for synthesis, only source is needed, the speaker copies everything for a complete environment).
- **Commands:**

```
cp -r ~/AI-HDL/tinyqv/test/* .  
cp -r ~/AI-HDL/tinyqv/verify/* .
```

Step 2.3: The AI-Assisted Synthesis Script

Instead of writing the Yosys TCL script from memory, the speaker uses **Google Gemini** to generate it.

- **The AI Prompt:**

The speaker types the following into the LLM:

```
"I want to run synthesis. Here is my list of files: peripheral.v , tt_wrapper.v ,  
[lists other files]. The top module is tt_wrapper . Write a Yosys script for this."
```

- **The AI Output (The Script):**

The AI generates a script block. The speaker copies this code.

- **Creating the File (`synth.ys`):**

1. **Command:** `nano synth.ys`
2. **Paste Content:**

```
# Read the Verilog Files
read_verilog -sv peripheral.v
read_verilog -sv tt_wrapper.v
# (Note: He ensures the Top Module files are included)

# Check Hierarchy
hierarchy -top tt_wrapper

# Synthesize
synth -top tt_wrapper

# Cleanup and Statistics
opt
clean
stat
```

3. **Save & Exit:** Press `Ctrl+O` (Write Out), `Enter` , `Ctrl+X` (Exit).

Step 2.4: Running Synthesis

- **Command:**

```
yosys synth.py
```

- **Visual Confirmation:** The terminal scrolls rapidly.
 - **Success Indicator:** The output stops at a section titled **"2.50. Printing statistics"**.
 - *Check:* Look for `Number of cells:` . If this number is > 0 and there are no "ERROR" messages above, your code is valid.
-

3. Phase 2: Physical Design with OpenLANE

Goal: Convert the synthesized code into a physical layout to generate PPA (Power, Performance, Area) metrics.

Step 3.1: Migrating to OpenLANE

OpenLANE runs inside Docker and expects projects to be in specific directories.

1. Navigate to OpenLANE Designs:

```
cd ~/AI-HDL/OpenLane/designs
```

2. Create Project Folder:

```
mkdir DP_1
cd DP_1
```

3. Import Files:

The speaker copies the *verified* files from the previous step into this new folder.

- **Command:** `cp ~/AI-HDL/DP_1/*.v .` (Copying all Verilog files).

Step 3.2: The `config.json` Configuration

This is the most critical step. OpenLANE is controlled by this file. One syntax error causes the flow to crash.

- **The AI Prompt:**

```
"I want to run PPA analysis using OpenLane. Here is my file list... Top module is
tt_wrapper . Create the config.json."
```

- **Creating the File:**

- **Command:** `nano config.json`
- **Paste Content (Exact Structure Shown):**

```
{
  "DESIGN_NAME": "tt_wrapper",
  "VERILOG_FILES": [
    "dir::peripheral.v",
    "dir::tt_wrapper.v"
  ],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 10.0,
  "FP_SIZING": "absolute",
  "DIE_AREA": "0 0 160 100",
  "PL_TARGET_DENSITY": 0.75
}
```

- **Critical Detail:** The speaker highlights the `dir::` prefix.
 - **Meaning:** `dir::` tells OpenLANE to look in the *current directory* inside the Docker container. Without this, it will fail to find your files.

Step 3.3: Running the Flow (The "Docker" Step)

1. Go to OpenLANE Root:

```
cd ~/AI-HDL/OpenLane
```

2. Enter Docker Container:

```
make mount
```

- *Visual Change:* The prompt changes to: `OpenLane Container (f3b...) : /openlane$`. You are now inside the tool.

3. Execute the Flow:

```
./flow.tcl -design DP_1
```

Step 3.4: Monitoring the Run

The terminal will display a sequence of steps. The speaker notes these stages:

1. **Synthesis:** (Yosys runs again to map to the specific SkyWater 130nm technology).
 2. **Floorplan:** (Defining the 160x100 chip boundary).
 3. **Placement:** (Placing standard cells).
 4. **CTS:** (Clock Tree Synthesis).
 5. **Routing:** (Connecting metal layers).
 6. **Signoff:** (Magic/KLayout checks).
- **Time:** This takes approximately 3-5 minutes on the demo machine.
 - **Success Message:** `[SUCCESS]: Flow Completed Without Fatal Errors.`
-

4. Phase 3: PPA Analysis & Reporting

Goal: Extract the numbers needed for the competition submission.

Step 4.1: Locating the Metrics File

The speaker navigates to the results folder *after* exiting Docker (or using a separate terminal).

- **Path:**

```
cd designs/DP_1/runs/  
ls -lt # (Lists runs by time, pick the latest one)  
cd <Run_Tag>/reports/
```

- **Target File:** `metrics.csv`

Step 4.2: Key Metrics Interpretation (Spreadsheet View)

The speaker opens the CSV (visualized in Excel/Sheets in the video) and highlights:

Column Name	Meaning	Target
<code>die_area</code>	Physical size of the chip.	Smaller is better.
<code>Total Internal Power</code>	Power consumed by logic.	Lower is better.
<code>wns (Worst Negative Slack)</code>	Timing margin.	Must be Positive.
<code>tns (Total Negative Slack)</code>	Sum of all timing violations.	Must be Zero.

- *Speaker Warning:* "If `wns` is negative (e.g., -2.34), your chip does **not** work at the requested speed. You must optimize your logic or lower the clock speed."

Step 4.3: Visualizing the Layout (GDSII)

To see the physical chip:

1. **Navigate:** `cd results/final/gds/`
2. **Command:** `klayout tt_wrapper.gds`
3. **Visual:** The screen shows the intricate metal layers. The speaker zooms in to show standard cells (NAND/NOR gates) placed in rows.

5. Troubleshooting & AI Tips (From Screen Context)

- **Prompt Logging:** The speaker has a sidebar open showing his chat history with Gemini. He reiterates that you must **save these logs**.
- **Error Handling:**
 - If Yosys fails: "Copy the exact ERROR line from the terminal."
 - Paste into AI: "I got this error in Yosys: [Error]. How do I fix it?"
 - *Speaker Insight:* "The AI usually tells you if you missed a file in the `read_verilog` list."

- **Path Issues:** If OpenLANE says "File not found", check the `config.json` . Ensure the `VERILOG_FILES` list matches the actual file names in the directory exactly (case-sensitive).